

Informe Final: Autonomous Software Factory

Autor: Diego Calixto Mesias Vélez

Introducción

El presente informe documenta la integración de un pipeline completo bajo el paradigma de *Autonomous Software Factory*. El objetivo principal es demostrar la transición fluida desde la concepción de un producto (Discovery), pasando por su desglose ágil (Delivery), hasta la implementación técnica guiada por especificaciones (Spec-Driven Development - SDD) y su posterior validación automatizada mediante un *Quality Agent*. El proyecto desarrollado, denominado citasalud-agenda, es un microservicio diseñado para gestionar reservas médicas, garantizando la integridad de los datos y previniendo colisiones de horarios mediante estrictos controles de concurrencia.

Análisis del Discovery Agent

En la fase inicial, se utilizó el *Discovery Agent* para procesar las entrevistas de conceptualización del producto CitaSalud. El agente analizó las necesidades del negocio (clínicas y pacientes) y generó el MVP Canvas.

Uso del agente: Se alimentó al modelo con los dolores principales del usuario: la pérdida de tiempo en agendamientos manuales y los errores por reservas duplicadas.

Resultados: El MVP Canvas definió como propuesta de valor central un sistema de agendamiento automatizado, determinando que la funcionalidad crítica para el primer lanzamiento debía ser un motor de reservas capaz de validar la disponibilidad de los profesionales de la salud en tiempo real y sin margen de error.

Análisis del Agile Delivery Team

Con los *outputs* del MVP Canvas como insumo, el *Agile Delivery Team Agent* se encargó de traducir la visión del producto en elementos de trabajo accionables.

Uso del agente: Se instruyó al agente para generar Épicas e Historias de Usuario bajo el marco de trabajo ágil, asegurando el cumplimiento de los criterios INVEST (Independiente, Negociable, Valiosa, Estimable, Pequeña y Testeable).

Resultados: Se definieron las historias de usuario principales. Para el alcance de este proyecto, destacó la siguiente:

Análisis del desarrollo con SDD

Para la implementación de la funcionalidad de reservas se aplicó *Spec-Driven Development* (SDD) utilizando la herramienta Spec-Kit.

Uso de Spec-Kit: Se generaron los artefactos fundamentales (spec.md, plan.md, tasks.md), los cuales fueron debidamente versionados en el repositorio bajo el directorio specs/001-agenda-citas/. El archivo spec.md se convirtió en el contrato único de verdad para el desarrollo.

Implementación: Se utilizó un *stack* basado en Java y Spring Boot. El desarrollo incluyó la creación del dominio (Reserva.java), el controlador (ReservaController.java) y la lógica de negocio (AgendaService.java).

Reto técnico: Durante la implementación, la especificación exigía el cumplimiento del criterio **FR-006 (Concurrencia)**. Esto requirió diseñar una arquitectura *thread-safe* en el servicio, abandonando las colecciones simples y aplicando bloqueos de sincronización (synchronized) para garantizar que las transacciones de validación e inserción fuesen atómicas.

Análisis del Quality Agent

Uso del agente: Se ejecutó el pipeline automatizado a través del script quality-gate.py.

Resultados y Resolución:

Bloqueo Inicial: En la primera iteración, el *gate* rechazó el código (EXIT 1). El reporte arrojó que el criterio FR-006 incumplía debido a la ausencia de pruebas concurrentes, dejando la cobertura en 79.0% y advirtiendo una vulnerabilidad *high* (java.lang.security.audit.unsynchronized-collection-access).

Corrección: Se refactorizó AgendaService.java implementando sincronización en los métodos mutadores. Adicionalmente, se desarrolló la prueba reservaFranjaOcupada_concurrente_rechaza en JUnit utilizando CountdownLatch y un ExecutorService de múltiples hilos para simular condiciones de estrés reales.

Aprobación Final: Tras la corrección, se volvió a ejecutar el agente.

El *Quality Gate* pasó a estado verde (**APROBADO**), certificando:

Pruebas: 129/129 pruebas superadas con una cobertura de JaCoCo del **85.0%**.

Seguridad: 0 vulnerabilidades críticas, 0 *high* y 0 secretos expuestos (evaluado vía Semgrep MCP).

Criterios: 5/5 criterios de aceptación cumplidos (FR-001 al FR-006).

Conclusiones

La metodología *Autonomous Software Factory* elimina la subjetividad en la fase de entrega. Al delegar la validación de la *Definition of Done* a un *Quality Gate* determinista,

la calidad del software (como el manejo de *race conditions*) se convierte en una barrera infranqueable de entrada, previniendo la acumulación de deuda técnica.

El enfoque Spec-Driven Development (SDD) demuestra un alto valor al mantener una trazabilidad directa desde el descubrimiento del MVP (Unidad 1) hasta la validación de pruebas unitarias (Unidad 3), asegurando que el código final responda estrictamente a una necesidad de negocio documentada.

Recomendaciones

Integración Continua (CI): Se recomienda portar los *scripts* del *Quality Agent* a un entorno de CI/CD (como GitHub Actions) para que el gate se ejecute automáticamente ante cada *Pull Request*, bloqueando fusiones de código que no cumplan la DoD.

Expansión de Reglas de Seguridad: Para futuras iteraciones de citasalud-agenda, se sugiere añadir reglas personalizadas en Semgrep dentro del agente para evaluar la correcta inyección de dependencias y el manejo de excepciones a nivel global del controlador.

Anexo

Repositorios de Agentes (Discovery, Agile Delivery, Quality):

<https://github.com/Mesias-prog/Proyecto-Final.git>